

数据库设计说明书

项目名称：火车站票务管理系统
课程：数据库原理与设计课程设计
数据库：MySQL 8.0
字符集：utf8mb4 / utf8mb4_unicode_ci

一、设计概述

1.1 设计目标

本系统的数据库设计需要满足以下要求：

- 数据完整性**：通过主键、外键、唯一索引等约束保证数据一致性
- 业务规则自动化**：利用触发器实现复杂业务规则（超员检查、自动座位更新）
- 统计查询效率**：使用存储过程封装复杂统计逻辑
- 扩展性与维护性**：符合规范化设计，降低数据冗余

1.2 设计原则

- ✔ 严格遵循**第三范式（3NF）**
- ✔ 原子性原则（字段不可再分）
- ✔ 主键与外键的合理使用
- ✔ 适当的索引优化查询性能
- ✔ 表和字段使用中文注释，提高可读性

1.3 数据库环境

配置项	值
数据库系统	MySQL 8.0
字符集	utf8mb4
排序规则	utf8mb4_unicode_ci
存储引擎	InnoDB（支持事务、外键）
表数量	7张业务表
触发器数量	2个
存储过程数量	3个

二、概念结构设计

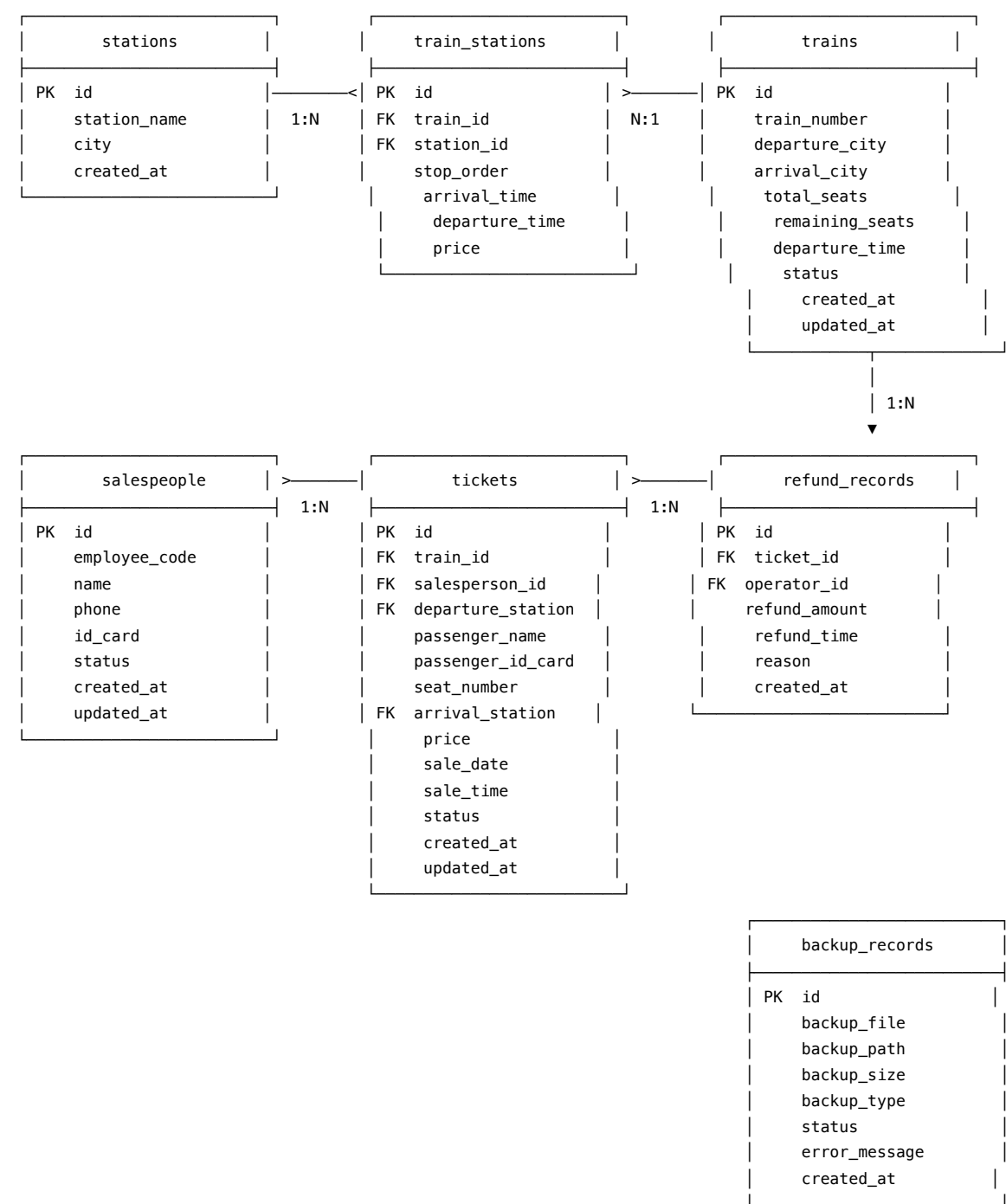
2.1 实体分析

通过需求分析，识别出以下实体：

实体名	中文名	说明
Train	车次	列车的基本信息，如车次号、出发城市等
Station	站点	火车站，包含站点名和所在城市

实体名	中文名	说明
TrainStation	车次站点	车次经停的站点，包含价格
Salesperson	业务员	售票员信息
Ticket	车票	销售的车票记录
RefundRecord	退票记录	退票的详细信息
BackupRecord	备份记录	数据备份的历史

2.2 E-R图



2.3 实体关系说明

关系	联系类型	说明
trains ↔ train_stations	1:N	一个车次包含多个经停站点
stations ↔ train_stations	1:N	一个站点可以被多个车次经停
trains ↔ tickets	1:N	一个车次有多张车票
stations ↔ tickets	1:N (×2)	站点作为出发站和到达站被引用
salespeople ↔ tickets	1:N	一个业务员售出多张票
tickets ↔ refund_records	1:N	一张票可以退票，产生记录

三、逻辑结构设计

3.1 关系模式

根据概念设计，转换得到以下关系模式（主键下划线标注）：

```
trains(<u>id</u>, train_number, departure_city, arrival_city,
      total_seats, remaining_seats, departure_time, status,
      created_at, updated_at)

stations(<u>id</u>, station_name, city, created_at)

train_stations(<u>id</u>, train_id, station_id, stop_order,
              arrival_time, departure_time, price, created_at)

salespeople(<u>id</u>, employee_code, name, phone, id_card,
            status, created_at, updated_at)

tickets(<u>id</u>, train_id, salesperson_id, departure_station_id,
        arrival_station_id, passenger_name, passenger_id_card,
        seat_number, price, sale_date, sale_time, status,
        created_at, updated_at)

refund_records(<u>id</u>, ticket_id, operator_id, refund_amount,
              refund_time, reason, created_at)

backup_records(<u>id</u>, backup_file, backup_path, backup_size,
              backup_type, status, error_message, created_at)
```

四、范式分析

4.1 范式定义

范式	要求
1NF（第一范式）	属性不可再分，原子性
2NF（第二范式）	非主属性完全依赖于主键（消除部分依赖）
3NF（第三范式）	非主属性之间不存在传递依赖

4.2 各表范式验证

4.2.1 trains表

项目	内容
候选键	id , train_number
主键	id
函数依赖	id → train_number, departure_city, arrival_city, total_seats, remaining_seats, departure_time, status, created_at, updated_at
范式	1NF  2NF  3NF 
说明	所有非主属性都直接依赖于 id , 属性之间无传递依赖

4.2.2 stations表

项目	内容
候选键	id , station_name
主键	id
函数依赖	id → station_name, city, created_at
范式	1NF  2NF  3NF 
说明	city 直接依赖于主键, 无传递依赖

4.2.3 train_stations表

项目	内容
候选键	id , (train_id, station_id)
主键	id
函数依赖	(train_id, station_id) → stop_order, arrival_time, departure_time, price
范式	1NF  2NF  3NF 
说明	所有属性都直接依赖于主键组合 price 等

4.2.4 salespeople表

项目	内容
候选键	id , employee_code
主键	id
函数依赖	id → employee_code, name, phone, id_card, status, created_at, updated_at
范式	1NF  2NF  3NF 
说明	候选键 employee_code 已添加UNIQUE约束

4.2.5 tickets表（重点）

项目	内容
候选键	(train_id, sale_date, seat_number) , id
主键	id
函数依赖	(train_id, sale_date, seat_number) → passenger_name, passenger_id_card, departure_station_id, arrival_station_id, seat_number
范式	1NF  2NF  3NF 

项目	内容
说明	添加 UNIQUE KEY uk_train_date_seat (train_id, sale_date, seat_number) 确保同一车次同一日期同一座位只能有一张有效票

4.2.6 refund_records表

项目	内容
候选键	id
主键	id
函数依赖	id → ticket_id, operator_id, refund_amount, refund_time, reason
范式	1NF  2NF  3NF 

4.2.7 backup_records表

项目	内容
候选键	id
主键	id
函数依赖	id → 所有属性
范式	1NF  2NF  3NF 

4.3 设计决策与说明

决策1：tickets表设计为整合方案

问题：为何不将tickets表拆分为"座位表"和"票表"？

方案	优点	缺点
拆分方案	座位可被复用	JOIN开销大，事务复杂
整合方案（采用）	事务原子性强，查询简洁	座位状态有冗余

理由：票务系统的核心操作是售票和退票，要求快速、原子完成。整合方案避免多表事务开销，且座位状态只是"已售/未售"的标记，不会造成数据不一致。

决策2：train_stations表嵌入price字段

问题：为何不将价格独立成一张price表？

方案	优点	缺点
独立价格表	相同区间可复用	JOIN开销大、维护复杂
嵌入train_stations（采用）	查询高效，票面直接获取	价格有少量重复

理由：车票价格由"出发站+车次+到达站"决定，嵌入业务表符合关系模型，且报价查询是高频操作。

决策3：refund_records表冗余存储refund_amount

问题：为何不在退票记录中通过JOIN计算金额？

方案	优点	缺点
计算字段	无冗余	历史规则变化后无法重现
冗余存储（采用）	可追溯历史金额	占用少量空间

理由：退票金额随时间可能变化（开车前/后），需要记录实际退票金额，保证财务可追溯。

五、物理结构设计

5.1 索引设计

表	索引名	类型	字段	用途
stations	PRIMARY	BTREE	id	主键
stations	uk_station_name	UNIQUE	station_name	站点名唯一
stations	idx_city	INDEX	city	按城市查询
trains	PRIMARY	BTREE	id	主键
trains	uk_train_number	UNIQUE	train_number	车次号唯一
trains	idx_departure	INDEX	departure_city	出发城市查询
trains	idx_arrival	INDEX	arrival_city	到达城市查询
train_stations	PRIMARY	BTREE	id	主键
train_stations	uk_train_station	UNIQUE	(train_id, station_id)	车次+站点唯一
train_stations	idx_train	INDEX	train_id	车次查询
train_stations	idx_station	INDEX	station_id	站点查询
salespeople	PRIMARY	BTREE	id	主键
salespeople	uk_employee_code	UNIQUE	employee_code	工号唯一
salespeople	idx_status	INDEX	status	按状态查询
tickets	PRIMARY	BTREE	id	主键
tickets	uk_train_date_seat	UNIQUE	(train_id, sale_date, seat_number)	同一座位唯一
tickets	idx_train_date	INDEX	(train_id, sale_date)	车次+日期查询
tickets	idx_salesperson	INDEX	salesperson_id	业务员查询
tickets	idx_sale_date	INDEX	sale_date	日期查询
tickets	idx_status	INDEX	status	状态查询

5.2 存储引擎选择

所有表使用 **InnoDB** 引擎，原因：

-  支持事务（ACID）
-  支持行级锁，提高并发性能
-  支持外键约束
-  支持MVCC（多版本并发控制）

六、业务规则实现

6.1 触发器设计

触发器1：售票前检查余票和座位

```
DELIMITER //
CREATE TRIGGER trg_before_ticket_sale
BEFORE INSERT ON tickets
FOR EACH ROW
BEGIN
    DECLARE v_remaining INT;
    DECLARE v_total INT;

    -- 检查车次是否存在
    SELECT total_seats, remaining_seats
    INTO v_total, v_remaining
    FROM trains WHERE id = NEW.train_id;

    -- 车次不存在则报错
    IF v_total IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = '车次不存在';
    END IF;

    -- 余票不足则报错
    IF v_remaining <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = '余票不足, 无法售票';
    END IF;
END//
DELIMITER ;
```

功能：在INSERT之前自动检查，确保不会超员售票。

触发器2：退票后自动恢复座位

```
DELIMITER //
CREATE TRIGGER trg_after_refund
AFTER UPDATE ON tickets
FOR EACH ROW
BEGIN
    -- 当车票从有效变为已退票时
    IF OLD.status = 1 AND NEW.status = 0 THEN
        -- 自动恢复车次座位
        UPDATE trains
        SET remaining_seats = remaining_seats + 1
        WHERE id = OLD.train_id AND remaining_seats < total_seats;
    END IF;
END//
DELIMITER ;
```

功能：在车票状态变为已退票时，自动增加该车次的剩余座位数。

6.2 存储过程设计

存储过程1：车次售票统计

名称：sp_train_sales_statistics(p_train_number, p_sale_date)

功能：统计指定车次在指定日期的售票情况

```

CREATE PROCEDURE sp_train_sales_statistics(
    IN p_train_number VARCHAR(20),
    IN p_sale_date DATE
)
BEGIN
    DECLARE v_train_id BIGINT;

    -- 查找车次
    SELECT id INTO v_train_id
    FROM trains WHERE train_number COLLATE utf8mb4_unicode_ci = p_train_number;

    IF v_train_id IS NULL THEN
        SELECT 'Train not found' AS error_message;
    ELSE
        -- 统计各区间售票
        SELECT
            tr.train_number,
            ds.station_name AS departure_station,
            as_st.station_name AS arrival_station,
            COUNT(tk.id) AS ticket_count,
            COALESCE(SUM(tk.price), 0) AS total_amount,
            tr.remaining_seats
        FROM trains tr
        LEFT JOIN tickets tk ON tr.id = tk.train_id
            AND tk.sale_date = p_sale_date AND tk.status = 1
        LEFT JOIN stations ds ON tk.departure_station_id = ds.id
        LEFT JOIN stations as_st ON tk.arrival_station_id = as_st.id
        WHERE tr.train_number COLLATE utf8mb4_unicode_ci = p_train_number
        GROUP BY tr.id, ds.id, as_st.id, tr.train_number, tr.remaining_seats;
    END IF;
END

```

业务价值：

- 帮助调度员实时掌握各车次销售情况
- 辅助决策是否需要加挂车厢
- 业务高峰期数据分析

存储过程2：业务员销售收入统计

名称： sp_salesperson_revenue(p_date)

功能： 统计指定日期各业务员的销售收入

```

CREATE PROCEDURE sp_salesperson_revenue(IN p_date DATE)
BEGIN
    SELECT
        sp.employee_code,
        sp.name AS salesperson_name,
        COUNT(tk.id) AS ticket_count,
        COALESCE(SUM(tk.price), 0) AS total_revenue
    FROM salespeople sp
    LEFT JOIN tickets tk ON sp.id = tk.salesperson_id
        AND DATE(tk.sale_time) = p_date AND tk.status = 1
    WHERE sp.status = 1
    GROUP BY sp.id, sp.employee_code, sp.name
    ORDER BY total_revenue DESC;
END

```

业务价值：

- 业务员绩效评估
- 销售业绩排名

- 财务报表数据来源

存储过程3：车次详细信息查询

名称： sp_train_details(p_train_number)

功能： 获取车次详细信息（含站点和价格）

七、数据完整性约束

7.1 实体完整性

通过主键约束保证：

表	主键	约束类型
trains	id	AUTO_INCREMENT
stations	id	AUTO_INCREMENT
train_stations	id	AUTO_INCREMENT
salespeople	id	AUTO_INCREMENT
tickets	id	AUTO_INCREMENT
refund_records	id	AUTO_INCREMENT
backup_records	id	AUTO_INCREMENT

7.2 参照完整性

通过外键约束保证：

表	外键字段	引用表	引用字段
train_stations	train_id	trains	id
train_stations	station_id	stations	id
tickets	train_id	trains	id
tickets	departure_station_id	stations	id
tickets	arrival_station_id	stations	id
tickets	salesperson_id	salespeople	id
refund_records	ticket_id	tickets	id
refund_records	operator_id	salespeople	id

7.3 用户自定义完整性

-  NOT NULL约束：业务必填字段
-  UNIQUE约束：站点名、车次号、工号、座位号
-  CHECK约束（应用层）：余票不能为负数等
-  触发器检查：售票前检查余票和座位占用

八、安全性与权限

8.1 用户权限

数据库创建了两个用户：

用户名	密码	权限
root	root123456	全部权限（应用直接使用）
trainuser	train123456	train_station_db的所有权限

8.2 连接配置

应用通过环境变量配置数据库连接：

```
SPRING_DATASOURCE_HOST: mysql
SPRING_DATASOURCE_PORT: 3306
SPRING_DATASOURCE_USERNAME: root
SPRING_DATASOURCE_PASSWORD: root123456
```

九、数据库初始化与维护

9.1 初始化顺序

```
01-schema.sql      # 创建数据库和表结构
02-triggers.sql     # 创建触发器
03-procedures.sql   # 创建存储过程
04-sample-data.sql  # 插入示例数据
```

通过Docker的 /docker-entrypoint-initdb.d 目录自动按字母顺序执行。

9.2 备份策略

- ✔ 完整备份（mysqldump）
- ✔ 历史备份记录表
- ✔ 一键恢复接口

十、设计总结

10.1 设计亮点

- 严格的3NF规范：7张表全部满足第三范式，消除了数据冗余和更新异常
- 触发器自动化：售票前业务规则检查、退票后座位恢复均自动完成
- 存储过程封装：复杂统计查询在数据库层完成，减轻应用负担
- 完善的索引：高频查询字段均有索引支撑
- 清晰的ER关系：7张表覆盖票务系统的完整业务
- 可追溯性：退票金额、备份历史均有完整记录

10.2 表数量与业务对应

业务需求	对应表
车次管理	trains

业务需求	对应表
站点管理	stations
车次站点配置	train_stations
业务员管理	salespeople
车票销售	tickets
退票管理	refund_records
数据备份	backup_records

10.3 范式统计

范式	表数量	占比
1NF	7	100%
2NF	7	100%
3NF	7	100%

结论：本数据库设计严格遵循第三范式，达到了规范化设计的标准。

附录：

- 完整SQL脚本
- 触发器脚本
- 存储过程脚本
- 示例数据